

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 - CODING CHALLENGE: ANSWERS

Name	K VENKATA VAMSI GOUD
Reg No	192472021
Course Code and Name	DSA0304 - NATURAL LANGUAGE PROCESSING
Assessment	Assessment Tool 3 - Coding Challenge (CO1)

Question 1: Intelligent Email and Password Validator

1. Problem Understanding

A registration system must accept an account only if the email address, password and mobile number all satisfy fixed security rules. Each field is checked separately and one of two fixed messages must be displayed. The constraints and the edge cases identified are listed below.

Field	Rule from the problem statement	Edge cases considered
Email	Starts with a letter; letters, digits, '.' and '_' before '@'; domain has only letters; extension is com, org, edu, net or in	Starts with a digit; double '@'; digit in domain (mail2.org); extension .xyz; empty string
Password	At least 8 characters with an uppercase letter, a lowercase letter, a digit and one of @ # \$ % & !	Exactly 7 characters; only one rule missing; empty password; non-string input
Mobile	Exactly 10 digits, first digit 6-9	11 digits; first digit 5; space inside the number; trailing newline; non-ASCII digits

2. Algorithm

- **Email and mobile:** Each field is validated by one pre-compiled regular expression, applied with `fullmatch()` so the whole string must satisfy the rule. Using `fullmatch()` instead of `'$'` also rejects a trailing newline.
- **Password:** Positive lookaheads `(?=.*[A-Z])`, `(?=.*[a-z])`, `(?=.*[0-9])`, `(?=.*[@#$%&!])` test the four character-class rules independently of their order, followed by `.{8,}` for the length. This is one pattern instead of five separate checks.
- **Feedback:** For a weak password, `password_gaps()` tests each rule separately and reports exactly which rules are missing.
- **Safety:** `[0-9]` is used instead of `\d`, because in Python `\d` also matches non-ASCII digits.

Complexity: Time is $O(n)$ for a string of length n , because every pattern uses only linear scans and has no nested quantifiers (so no catastrophic backtracking). The four lookaheads multiply the work only by a constant. Extra space is $O(1)$, and patterns are compiled once.

3. Code Implementation

```
import re

# ----- Pre-compiled patterns (compiled once, reused for every call) -----
EMAIL_RE = re.compile(r"[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(?:com|org|edu|net|in)", re.I)
PASSWORD_RE = re.compile(
    r"(?=.*[A-Z])"      # at least one uppercase letter
    r"(?=.*[a-z])"      # at least one lowercase letter
    r"(?=.*[0-9])"      # at least one digit
    r"(?=.*[@#$%&!])"   # at least one special character
    r".{8,}"           # minimum 8 characters
    re.S)              # minimum 8 characters
MOBILE_RE = re.compile(r"[6-9][0-9]{9}") # [0-9] (not \d) so Unicode digits are rejected

PASSWORD_RULES = [
    ("minimum 8 characters", re.compile(r".{8,}", re.S)),
    ("an uppercase letter", re.compile(r"[A-Z]")),
    ("a lowercase letter", re.compile(r"[a-z]")),
    ("a digit", re.compile(r"[0-9]")),
    ("a special character (@ # $ % & !)", re.compile(r"[@#$%&!]")),
]

def _clean(value):
    """Reject non-string input; fullmatch() below checks the whole string."""
    if not isinstance(value, str):
        raise TypeError("Input must be a string")
    return value

def validate_email(email):
    return bool(EMAIL_RE.fullmatch(_clean(email)))

def validate_password(password):
    return bool(PASSWORD_RE.fullmatch(_clean(password)))

def password_gaps(password):
    """Return the list of unmet password rules (used for helpful feedback)."""
    return [text for text, rx in PASSWORD_RULES if not rx.search(password)]

def validate_mobile(mobile):
    return bool(MOBILE_RE.fullmatch(_clean(mobile)))

def check(email, password, mobile):
    print(f"Input : email={email!r}, password={password!r}, mobile={mobile!r}")
    try:
        e, p, m = validate_email(email), validate_password(password), validate_mobile(mobile)
    except TypeError as err:
        print(f"Error : {err}\n")
        return
    print("Email : ", "Valid Email" if e else "Invalid Email")
    if p:
        print("Password : Strong Password")
    else:
        print("Password : Weak Password (missing: " + ", ".join(password_gaps(password)) + ")")
    print("Mobile : ", "Valid Mobile Number" if m else "Invalid Mobile Number")
    print()
```

```

if __name__ == "__main__":
    tests = [
        ("john.doe_21@gmail.com", "Secure@123", "9876543210"),
        ("ljohn@gmail.com", "password", "5876543210"),
        ("anita@mail2.org", "Abcdefl", "98765432100"),
        ("ravi@college.edu", "ABCD@1234", "6000000000"),
        ("user@site.xyz", "Passw0rd!", "98765 43210"),
        ("a_b.c@uni.in", "aB3$aB3$", "9999999999"),
        ("user@@site.com", "Pass word1!", "9876543210\n"),
        ("", "", ""),
        ("test@domain.net", "12345678", "9876543210"),
    ]
    for t in tests:
        check(*t)

```

4. Test Case Handling and Output

No.	Test (email / password / mobile)	Email	Password	Mobile
1	john.doe_21@gmail.com / Secure@123 / 9876543210	Valid	Strong	Valid
2	ljohn@gmail.com / password / 5876543210	Invalid	Weak	Invalid
3	anita@mail2.org / Abcdefl / 98765432100	Invalid	Weak	Invalid
4	ravi@college.edu / ABCD@1234 / 6000000000	Valid	Weak	Valid
5	user@site.xyz / Passw0rd! / 98765 43210	Invalid	Strong	Invalid
6	a_b.c@uni.in / aB3\$aB3\$ / 9999999999	Valid	Strong	Valid
7	user@@site.com / Pass word1! / trailing newline	Invalid	Strong	Invalid
8	empty email, password and mobile	Invalid	Weak	Invalid
9	password given as a number (12345678)	Error: Input must be a string		

Program output:

```

Input  : email='john.doe_21@gmail.com', password='Secure@123', mobile='9876543210'
Email   : Valid Email
Password : Strong Password
Mobile  : Valid Mobile Number

Input  : email='ljohn@gmail.com', password='password', mobile='5876543210'
Email   : Invalid Email
Password : Weak Password (missing: an uppercase letter, a digit, a special character (@ # $ % & !))
Mobile  : Invalid Mobile Number

```

```

Input   : email='anita@mail2.org', password='Abcdefl', mobile='98765432100'
Email   : Invalid Email
Password : Weak Password (missing: minimum 8 characters, a special character (@ # $ % & !))
Mobile  : Invalid Mobile Number

Input   : email='ravi@college.edu', password='ABCD@1234', mobile='6000000000'
Email   : Valid Email
Password : Weak Password (missing: a lowercase letter)
Mobile  : Valid Mobile Number

Input   : email='user@site.xyz', password='Passw0rd!', mobile='98765 43210'
Email   : Invalid Email
Password : Strong Password
Mobile  : Invalid Mobile Number

Input   : email='a_b.c@uni.in', password='aB3$aB3$', mobile='9999999999'
Email   : Valid Email
Password : Strong Password
Mobile  : Valid Mobile Number

Input   : email='user@@@site.com', password='Pass word1!', mobile='9876543210\n'
Email   : Invalid Email
Password : Strong Password
Mobile  : Invalid Mobile Number

Input   : email="", password="", mobile=""
Email   : Invalid Email
Password : Weak Password (missing: minimum 8 characters, an uppercase letter, a lowercase letter, a digit, a
special character (@ # $ % & !))
Mobile  : Invalid Mobile Number

Input   : email='test@domain.net', password=12345678, mobile='9876543210'
Error   : Input must be a string

```

5. Competitive Performance and Justification

Every check is a single linear pass over a short string, so the validator handles thousands of registrations per second. Test 4 shows that the lookahead design reports a single missing rule (lowercase letter), tests 2, 3 and 8 show multiple missing rules, and tests 5 and 7 show that the three fields are judged independently. Test 9 shows that wrong input types are reported through exception handling and the program does not crash.

Question 2: Deterministic Finite Automaton (DFA) Simulator

1. Problem Understanding

A DFA is defined by the five-tuple $M = (Q, \Sigma, \delta, q_0, F)$. The simulator must accept this description, accept several input strings, show every state transition and display Accepted or Rejected. A string is accepted if and only if the DFA is in a final state after the last symbol is read.

Constraint / edge case	Handling in the program
Description must be a valid DFA	<code>_validate()</code> checks that the initial state and final states belong to Q , that every (state, symbol) pair has exactly one transition, and that all transitions use known states and symbols
Symbol outside the alphabet	Reading stops and the string is Rejected, with an error message naming the symbol
Empty string	Accepted only if the initial state is also a final state
Multiple strings	Any number of strings can be tested against the same DFA
Input of the DFA description	Built in code, or entered interactively with the <code>--interactive</code> option

Note on the sample in the question: for the input `abaab` (5 symbols) a correct path has 6 states, because every symbol causes one transition. The sample path shown in the question has only 5 states, so it is shortened. The simulator prints the complete path $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_1 \rightarrow q_1 \rightarrow q_2$, and the result Accepted is the same.

2. Algorithm

- **Data structure:** The transition function is stored as a dictionary $\{(state, symbol): next_state\}$, which gives $O(1)$ lookup per step.
- **Simulation:** Start at q_0 ; for each input symbol, replace the current state by $\delta(state, symbol)$; after the last symbol, test whether the state is in the set of final states F (a Python set, $O(1)$ membership).
- **Example automata:** The DFA for 'strings ending with `ab`' has three states: q_0 (no progress), q_1 (last symbol is `a`) and q_2 (last two symbols are `ab`, final). A second DFA (even number of `1`s) shows that the simulator is general.

Complexity: Simulation takes $O(n)$ time for a string of length n , with one dictionary lookup per symbol. Extra space is $O(1)$ when the path is not stored (`keep_path=False`) and $O(n)$ when the path is printed. Validation of the description takes $O(|Q| \times |\Sigma|)$, done once.

3. Code Implementation

```
import sys
import time

class DFA:
    """Deterministic Finite Automaton M = (Q, Sigma, delta, q0, F)."""

    def __init__(self, states, alphabet, transitions, start, finals):
        self.states, self.alphabet = set(states), set(alphabet)
```

```

self.delta, self.start, self finals = dict(transitions), start, set(finals)
self._validate()

def _validate(self):
    if self.start not in self.states:
        raise ValueError(f"Initial state {self.start!r} is not in the state set")
    if not self finals <= self.states:
        raise ValueError(f"Final states {self finals - self.states} are not in the state set")
    for q in self.states:          # DFA must be complete and deterministic
        for a in self.alphabet:
            if (q, a) not in self.delta:
                raise ValueError(f"Missing transition for ({q}, {a})")
    for (q, a), r in self.delta.items():
        if q not in self.states or a not in self.alphabet or r not in self.states:
            raise ValueError(f"Invalid transition ({q}, {a}) -> {r}")

def run(self, string, keep_path=True):
    """Simulate the DFA. Time O(n), extra space O(1) (O(n) only if the path is kept)."""
    state, path = self.start, [self.start]
    for ch in string:
        if ch not in self.alphabet:
            return False, path, f"symbol {ch!r} is not in the alphabet"
        state = self.delta[(state, ch)]
        if keep_path:
            path.append(state)
    return state in self finals, path, None

def test(self, string):
    accepted, path, error = self.run(string)
    shown = string if string else "(empty string)"
    print(f"Input      : {shown}")
    print(f"Transition Path : " + " -> ".join(path))
    if error:
        print(f"Error      : {error}")
    print(f"Result      :", "Accepted" if accepted else "Rejected")
    print()
    return accepted

def read_dfa_from_user():
    """Interactive input of the DFA description."""
    states = input("States (space separated) : ").split()
    alphabet = input("Input alphabet (space separated): ").split()
    start = input("Initial state : ").strip()
    finals = input("Final state(s) (space separated) : ").split()
    print("Enter transitions as 'state symbol next_state' (blank line to finish):")
    delta = {}
    while True:
        line = input().split()
        if not line:
            break
        if len(line) != 3:
            print("Format error, expected 3 items - ignored"); continue
        delta[(line[0], line[1])] = line[2]
    return DFA(states, alphabet, delta, start, finals)

def ends_with_ab():

```

```

return DFA(["q0", "q1", "q2"], ["a", "b"],
           {("q0", "a"): "q1", ("q0", "b"): "q0",
            ("q1", "a"): "q1", ("q1", "b"): "q2",
            ("q2", "a"): "q1", ("q2", "b"): "q0"}, "q0", ["q2"])

def even_number_of_ones():
    return DFA(["e", "o"], ["0", "1"],
               {("e", "0"): "e", ("e", "1"): "o", ("o", "0"): "o", ("o", "1"): "e"}, "e", ["e"])

def print_table(dfa):
    cols = sorted(dfa.alphabet)
    print("Transition table:")
    print(" " + f'{"State":<10}' + "".join(f'{"c":<8}' for c in cols))
    for q in sorted(dfa.states):
        mark = (">" if q == dfa.start else " ") + ("*" if q in dfa.finals else " ")
        print(f'{mark} {"q":<10}' + "".join(f'{"dfa.delta[(q, c)":<8}' for c in cols))
    print()

if __name__ == "__main__":
    if "--interactive" in sys.argv:
        dfa = read_dfa_from_user()
        print_table(dfa)
        while True:
            s = input("Enter string (or 'quit'): ")
            if s == "quit": break
            dfa.test(s)
    else:
        print("##### DFA 1 : strings over {a,b} ending with 'ab' #####")
        d1 = ends_with_ab()
        print_table(d1)
        for s in ["abaab", "ab", "abb", "bbaab", "a", "", "abc"]:
            d1.test(s)

        print("##### DFA 2 : binary strings with an even number of 1s #####")
        d2 = even_number_of_ones()
        for s in ["1010", "111", "0000"]:
            d2.test(s)

        print("##### Invalid DFA description (error handling) #####")
        try:
            DFA(["q0", "q1"], ["a"], {("q0", "a"): "q1"}, "q0", ["q1"])
        except ValueError as err:
            print("Rejected description:", err, "\n")

        print("##### Performance on a large input (n = 1,000,000) #####")
        big = "ab" * 500_000 + "ab"
        big = big[1_000_000 - 2] + "ab"
        t = time.perf_counter()
        accepted, _, _ = d1.run(big, keep_path=False)
        print(f'Length {len(big):,} -> {"Accepted" if accepted else "Rejected"} "
              f"in {time.perf_counter() - t:.3f} s")

```

4. Test Case Handling and Output

Test string	Reason / language	Expected	Program result
abaab	Ends with ab (sample)	Accepted	Accepted
ab	Shortest accepted string	Accepted	Accepted
abb	Ends with bb	Rejected	Rejected
bbaab	Leading b symbols, then ends with ab	Accepted	Accepted
a	Only half of the pattern	Rejected	Rejected
(empty)	q0 is not final	Rejected	Rejected
abc	c is not in the alphabet	Rejected with error	Rejected with error
1010 / 111 / 0000	Even number of 1s (DFA 2)	Accepted / Rejected / Accepted	Same
Incomplete DFA	Transition (q1, a) missing	Description rejected	Description rejected

Program output:

```
##### DFA 1 : strings over {a,b} ending with 'ab' #####
```

```
Transition table:
```

```
State   a   b
-> q0    q1  q0
  q1    q1  q2
*q2     q1  q0
```

```
Input      : abaab
```

```
Transition Path : q0 -> q1 -> q2 -> q1 -> q1 -> q2
```

```
Result      : Accepted
```

```
Input      : ab
```

```
Transition Path : q0 -> q1 -> q2
```

```
Result      : Accepted
```

```
Input      : abb
```

```
Transition Path : q0 -> q1 -> q2 -> q0
```

```
Result      : Rejected
```

```
Input      : bbaab
```

```
Transition Path : q0 -> q0 -> q0 -> q1 -> q1 -> q2
```

```
Result      : Accepted
```

```
Input      : a
```

```
Transition Path : q0 -> q1
```

```
Result      : Rejected
```

```
Input      : (empty string)
```

```
Transition Path : q0
```

```
Result      : Rejected
```

```
Input      : abc
```

```
Transition Path : q0 -> q1 -> q2
```

```
Error      : symbol 'c' is not in the alphabet
Result     : Rejected

##### DFA 2 : binary strings with an even number of 1s #####
Input      : 1010
Transition Path : e -> o -> o -> e -> e
Result     : Accepted

Input      : 111
Transition Path : e -> o -> e -> o
Result     : Rejected

Input      : 0000
Transition Path : e -> e -> e -> e -> e
Result     : Accepted

##### Invalid DFA description (error handling) #####
Rejected description: Missing transition for (q1, a)

##### Performance on a large input (n = 1,000,000) #####
Length 1,000,000 -> Accepted in 0.225 s
```

5. Competitive Performance and Justification

The simulator processed a string of 1,000,000 symbols in a fraction of a second (the measured time is shown at the end of the output), because it performs exactly one table lookup per symbol and stores no path in this mode. The tracing of abaab by hand ($q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_1 \rightarrow q_1 \rightarrow q_2$) agrees with the program, which verifies the transition table. Invalid descriptions and invalid symbols are caught, so only correct automata are simulated.

Question 3: Smart Pattern Matching Engine

1. Problem Understanding

A text search engine must find dates, phone numbers, hashtags, mentions, and words by prefix or suffix, and let the user choose the search from a menu. The program also provides Word Search, because it is listed in the requirements even though it is not in the sample menu (option 7). Constraints and edge cases:

Search	Requirement / constraint	Edge cases considered
Date	dd/mm/yyyy format, as in 12/09/2026	Month 13; 31/04 and 29/02/2027 (impossible dates); leap year 29/02/2028; separators - and .
Phone	10-digit Indian mobile number starting with 6-9	Optional +91; 11-digit numbers; first digit 5
Hashtag	# followed by a word	'price#5' (not a hashtag); a lone '#'
Mention	@ followed by a username	An email address such as test@site.com must not be treated as a mention
Prefix / Suffix / Word	Matches at word level, case-insensitive	Empty term; special characters in the term; invalid menu choice

2. Algorithm

- **Core method:** Each search is a compiled regular expression scanned over the text once with `finditer()/findall()`, so the work is linear in the text size.
- **Patterns:** Lookbehind (`(?<![0-9])`) and lookahead (`(?![0-9])`) make sure a phone number or date is not part of a longer digit sequence; `(?<![\w.])` before '@' excludes emails. A backreference `\2` forces the same separator in a date, so 12/09-2026 is rejected.
- **Two-step date check:** The regular expression finds the shape of a date; `valid_date()` then checks the days of the month and leap years (a year divisible by 4, except century years not divisible by 400).
- **Safe user input:** The user's term goes through `re.escape()` and is limited to letters and digits, which prevents pattern injection and errors from characters like '(' or '*'.
- **Menu design:** `menu_loop()` takes the input function as a parameter, so the same code runs interactively with `input()` and in the scripted demonstration below.

Complexity: Date, phone, hashtag, mention, prefix and word searches take $O(n)$ time for a text of n characters. The suffix pattern `\w*term\b` has to try each word start, so it is $O(n)$ for normal text but is the slowest search (it can be quadratic in the length of one very long word). Space is $O(m)$ for the m matches returned.

3. Code Implementation

```
import re
import time

# ----- Compiled patterns (all linear-time: no nested quantifiers) -----
DATE_RE = re.compile(r"(?<![0-9])(0?[1-9]|[12][0-9]|3[01])([/-])(0?[1-9]|1[0-2])\2([0-9]{4})(?![0-9])")
```

```

PHONE_RE = re.compile(r"(?![0-9])(?:\+91[\s-]?)?[6-9][0-9]{9}(?![0-9])")
HASHTAG_RE = re.compile(r"(?![\w&])#[A-Za-z_]\w*")
MENTION_RE = re.compile(r"(?![\w.])@[A-Za-z_]\w*")

def valid_date(m):
    """Check the calendar (e.g. 31/04/2026 and 29/02/2027 are rejected)."""
    d, mo, y = int(m.group(1)), int(m.group(3)), int(m.group(4))
    days = [31, 29 if (y % 4 == 0 and (y % 100 != 0 or y % 400 == 0)) else 28,
            31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
    return d <= days[mo - 1]

def search_date(text):
    return [m.group(0) for m in DATE_RE.finditer(text) if valid_date(m)]

def search_phone(text):
    return [m.group(0) for m in PHONE_RE.finditer(text)]
def search_hashtag(text):
    return HASHTAG_RE.findall(text)
def search_mention(text):
    return MENTION_RE.findall(text)

def _word(term):
    term = term.strip()
    if not term or not term.isalnum():
        raise ValueError("Enter a single non-empty word (letters/digits only)")
    return re.escape(term)

def search_word(text, term):
    return re.findall(rf"\b{_word(term)}\b", text, re.I)

def search_prefix(text, term):
    return re.findall(rf"\b{_word(term)}\w*", text, re.I)

def search_suffix(text, term):
    return re.findall(rf"\w*{_word(term)}\b", text, re.I)

MENU = """
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
0. Exit"""

def show(title, results):
    print(f"{title}: " + (" ".join(results) if results else "No match found") +
          f"  [{len(results)} match(es)]")

def menu_loop(text, read=input):
    """Menu-driven interface; `read` can be replaced to feed scripted input."""
    while True:
        print(MENU)
        choice = read("Enter your choice: ").strip()
        print(f"> {choice}")
        try:
            if choice == "1": show("Dates", search_date(text))

```

```

elif choice == "2": show("Phone numbers", search_phone(text))
elif choice == "3": show("Hashtags", search_hashtag(text))
elif choice == "4": show("Mentions", search_mention(text))
elif choice in ("5", "6", "7"):
    term = read("Enter search term: ")
    print(f"> {term}")
    fn = {"5": search_prefix, "6": search_suffix, "7": search_word}[choice]
    show({"5": "Prefix matches", "6": "Suffix matches", "7": "Word matches"}[choice],
        fn(text, term))
elif choice == "0":
    print("Thank you. Exiting."); break
else:
    print("Invalid choice. Please select 0-7.")
except ValueError as err:
    print("Error:", err)

SAMPLE = """Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing"""

if __name__ == "__main__":
    print("INPUT TEXT:\n" + SAMPLE)
    scripted = iter(["1", "2", "3", "4", "5", "nat", "6", "ing", "7", "language",
                    "5", "", "9", "0"])
    menu_loop(SAMPLE, read=lambda prompt: next(scripted))

    print("\n===== EDGE CASES =====")
    edge = ("Dates: 31/04/2026 29/02/2027 29/02/2028 5-6-2025 12/13/2026 "
            "Phones: +91 9876543210 5876543210 98765432101 "
            "Tags: #AI_2026 price#5 # empty "
            "Mentions: @user_1 mail me at test@site.com")
    print(edge)
    show("Dates", search_date(edge))
    show("Phones", search_phone(edge))
    show("Hashtags", search_hashtag(edge))
    show("Mentions", search_mention(edge))

    print("\n===== PERFORMANCE (about 3.4 MB of text) =====")
    big = (SAMPLE + "\nprocessing nation natural 31/12/2025 #tag @user 9123456789\n") * 25_000
    print(f"Text size: {len(big):,} characters")
    for name, fn in [("Date", search_date), ("Phone", search_phone), ("Hashtag", search_hashtag),
                    ("Mention", search_mention), ("Prefix 'nat'", lambda t: search_prefix(t, "nat")),
                    ("Suffix 'ing'", lambda t: search_suffix(t, "ing"))]:
        t0 = time.perf_counter()
        n = len(fn(big))
        print(f'{name:<14}: {n:>7}, matches in {time.perf_counter() - t0:.3f} s')

```

4. Test Case Handling and Output

Input	Expected	Result
Menu 1 (sample text)	12/09/2026	12/09/2026
Menu 2	9876543210	9876543210

Input	Expected	Result
Menu 3 / Menu 4	#NLP / @OpenAI	#NLP / @OpenAI
Menu 5, term 'nat'	natural	natural
Menu 6, term 'ing'	Meeting, processing	Meeting, processing
Menu 7, term 'language'	language	language
Menu 5 with empty term	Error message	Error message
Menu choice 9	Invalid choice message	Invalid choice message
31/04/2026, 29/02/2027, 12/13/2026	Rejected (impossible dates)	Rejected
29/02/2028, 5-6-2025	Accepted	Accepted
+91 9876543210; 5876543210; 98765432101	Only the first is a phone number	Only the first
price#5; test@site.com	Not a hashtag; not a mention	Excluded

Program output:

```

INPUT TEXT:
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
0. Exit
> 1
Dates: 12/09/2026 [1 match(es)]

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
0. Exit
> 2
Phone numbers: 9876543210 [1 match(es)]

1. Search Date

```

2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
0. Exit
> 3
Hashtags: #NLP [1 match(es)]

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
0. Exit
> 4
Mentions: @OpenAI [1 match(es)]

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
0. Exit
> 5
> nat
Prefix matches: natural [1 match(es)]

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
0. Exit
> 6
> ing
Suffix matches: Meeting, processing [2 match(es)]

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
0. Exit
> 7
> language
Word matches: language [1 match(es)]

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
0. Exit
> 5
>
Error: Enter a single non-empty word (letters/digits only)

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
0. Exit
> 9
Invalid choice. Please select 0-7.

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
0. Exit
> 0
Thank you. Exiting.

===== EDGE CASES =====

Dates: 31/04/2026 29/02/2027 29/02/2028 5-6-2025 12/13/2026 Phones: +91 9876543210 5876543210 98765432101 Tags: #AI_2026 price#5 # empty Mentions: @user_1 mail me at test@site.com
Dates: 29/02/2028, 5-6-2025 [2 match(es)]
Phones: +91 9876543210 [1 match(es)]
Hashtags: #AI_2026 [1 match(es)]
Mentions: @user_1 [1 match(es)]

===== PERFORMANCE (about 3.4 MB of text) =====

Text size: 3,450,000 characters
Date : 50,000 matches in 0.323 s
Phone : 50,000 matches in 0.224 s
Hashtag : 50,000 matches in 0.123 s
Mention : 50,000 matches in 0.145 s
Prefix 'nat' : 75,000 matches in 0.125 s
Suffix 'ing' : 75,000 matches in 0.445 s

5. Competitive Performance and Justification

To test large inputs, the sample text was repeated into a text of about 3.4 million characters containing 50,000 dates, phones, hashtags and mentions. Each search finished in well under one second (timings are shown at the end of the output), and the match counts are exactly the expected values, which confirms both speed and correctness. The suffix search is the slowest of the six, as predicted in the complexity analysis. The hidden-case risks (impossible dates, numbers inside longer digit strings, emails mistaken for mentions, empty or special-character search terms) are all handled and shown in the edge-case output.

Conclusion

The three programs use regular expressions and finite state automata with linear-time algorithms, are modular, validate their inputs, handle errors, and were tested with valid, invalid, edge and large-input cases. The source code, sample outputs and a README can be uploaded to the GitHub repository as the instructions require.